

1)

A interação entre o VFS (Virtual Filesystem) e as implementações de sistemas de arquivos concretos ocorre por meio de uma camada de abstração que funciona como uma "cola" dentro do kernel, permitindo que diferentes sistemas coexistam e interoperem. O VFS define interfaces conceituais básicas e um modelo de arquivo comum que todos os sistemas de arquivos devem suportar. Quando um programa no espaço do usuário executa uma chamada de sistema padrão, como `write()`, essa solicitação é recebida primeiramente por uma função genérica do VFS, e não pelo sistema de arquivos diretamente.

Para concretizar a operação, o VFS utiliza estruturas de dados que contêm ponteiros para funções, conhecidas como tabelas de operações. O VFS invoca o método específico implementado pelo sistema de arquivos que está montado naquele ponto (por exemplo, a função de escrita específica do ext3 ou do ext4), delegando a ele a tarefa de lidar com os detalhes de implementação na mídia física. Dessa forma, o kernel consegue trabalhar de maneira uniforme com qualquer sistema de arquivos, exigindo apenas que as implementações concretas (inclusive aquelas que não são nativas do Unix, como o FAT) adaptem suas estruturas de dados em memória para corresponder à interface abstrata esperada pelo VFS.

2)

O inode representa todas as informações (metadados) necessárias pelo kernel para manipular um arquivo ou diretório, exceto o seu nome. Ele contém dados como permissões de acesso, tamanho, proprietário e timestamps. Em sistemas Unix, ele é lido diretamente do disco; em sistemas sem inodes nativos, ele é construído na memória pelo driver do sistema de arquivos.

3 Exemplos de campos da estrutura inode:

1. **i_size**: O tamanho do arquivo em bytes.
2. **i_uid**: O ID do usuário (User ID) do proprietário do arquivo.
3. **i_atime**: O timestamp do último acesso ao arquivo.

3 Exemplos de operações do inode :

1. **create()**: Invocado pelas chamadas de sistema **open()** e **creat()** para criar um novo inode (arquivo) associado a uma entrada de diretório (dentry).
2. **mkdir()**: Chamado para criar um novo diretório com um modo inicial específico.
3. **lookup()**: Procura em um diretório por um inode que corresponda a um nome de arquivo específico contido em um dentry.

3)

O objeto dentry (Directory Entry) representa um componente específico em um caminho de arquivo (path). O VFS trata diretórios como arquivos, e o dentry é usado para facilitar a busca de caminhos. Por exemplo, no caminho `"/bin/vi"`, tanto `"/"`, quanto `"bin"` e `"vi"` são objetos dentry. Eles ligam o nome do arquivo (string) ao seu inode correspondente e não são armazenados em disco, sendo criados dinamicamente na memória.

3 Exemplos de campos da estrutura dentry:

1. **d_inode**: Ponteiro para o inode associado a este dentry.

2. **d_parent**: Ponteiro para o objeto dentry do diretório pai.
3. **d_name**: O nome da entrada do diretório (ex: o nome do arquivo).

3 Exemplos de operações do dentry:

1. **d_compare()**: Usado para comparar dois nomes de arquivos. É crucial para sistemas de arquivos que não diferenciam maiúsculas de minúsculas (case-insensitive), como o FAT, que precisam sobrescrever a comparação padrão de strings.
2. **d_delete()**: Chamado quando a contagem de uso (d_count) do dentry chega a zero.
3. **d_revalidate()**: Determina se o objeto dentry ainda é válido. A maioria dos sistemas define como NULL, mas é usado quando a validade precisa ser checada antes do uso.

Estado do Dentry: Um dentry válido pode estar em um de três estados:

- **Used (Em uso)**: Corresponde a um inode válido (d_inode aponta para um inode) e possui usuários ativos (d_count > 0). Não pode ser descartado.
- **Unused (Não usado)**: Corresponde a um inode válido, mas o VFS não o está usando ativamente no momento (d_count == 0). Ele é mantido na memória (cache) para o caso de ser necessário novamente, mas pode ser descartado se houver falta de memória.
- **Negative (Negativo)**: Não está associado a um inode válido (d_inode é NULL). Isso ocorre quando um caminho é resolvido, mas o arquivo não existe. O dentry negativo é mantido para agilizar futuras buscas falhas pelo mesmo arquivo inexistente.

Cache do Dentry: Como a resolução de caminhos e a leitura de diretórios são operações custosas (leitura de disco, processamento de strings), o kernel armazena os objetos dentry na dcache.

- Isso acelera buscas futuras (localidade temporal e espacial).
- A dcache consiste em listas de dentries "em uso", uma lista LRU (Least Recently Used) para dentries não usados/negativos (para descarte eficiente) e uma tabela de hash para resolução rápida de caminhos em objetos dentry. A dcache também ajuda a manter os inodes associados na memória (icache).